

TRPO 实验报告

LAMDA-5

June 16, 2026

Abstract

本报告实现并分析了 Trust Region Policy Optimization (TRPO) 算法 [1]。我们完成了 `compute_advantages_from_traj`、`compute_advantages_from_rollout` 和 `_update_policy` 三部分代码补全，并在 Humanoid-v5、HalfCheetah-v5 和 Walker2d-v5 三个 MuJoCo 连续控制环境中进行了系统性实验。实验对比了 rollout 与 trajectory 两种采样方式，以及 advantage normalization 对训练稳定性的影响。结果表明：rollout 在样本利用率 (100%) 上优于 trajectory (62%–71%)，在 HalfCheetah 上最终回报明显更高；trajectory 在 Humanoid 等高维任务中能通过保留完整 episode 边界获得更高质量的 advantage 估计。Advantage normalization 对数值稳定性至关重要——在 Walker2d 中，关闭归一化后 surrogate objective 出现量级达 10^{33} 的非有限值，导致策略学习几乎失败。

1 实验设置

本实验在 Humanoid-v5、HalfCheetah-v5 和 Walker2d-v5 三个 MuJoCo Gymnasium v5 环境上比较两种采样方式 (rollout / trajectory) 与是否做 advantage normalization 的影响。主要超参数见表 1。

Table 1: 实验超参数设置

超参数	值
并行环境数	8
每 epoch 交互步数 (rollout)	2000 (共 $8 \times 2000 = 16000$ 个 transition)
轨迹条数 (trajectory)	10
最大 episode 长度	1000
总训练 epoch 数	5000
折扣因子 γ	0.99
GAE 参数 λ	0.95
KL 约束阈值 δ	0.01
共轭梯度迭代次数	10
Critic 优化器	Adam, 学习率 10^{-3}
Critic 更新轮数 / 批大小	5 / 256
策略/价值网络结构	2 层 MLP (64, 64), ReLU 激活; 动作经 Tanh 压缩至动作空间

表 2 汇总了各配置下的关键指标。最终测试回报取训练末期均值; trajectory 模式总交互步数与 rollout 不同, 所报数值为相近交互步数处的测试结果, 以 † 标注。

Table 2: 各配置实验结果汇总

环境	采样模式	Advan Norm	最终测试回报	样本利用率	平均有效步数/epoch
Humanoid-v5	rollout	✓	1083	100%	16000
	rollout		1085	100%	16000
	trajectory	✓	$\sim 655^{\dagger}$	69.0%	~ 885
HalfCheetah-v5	rollout	✓	4276	100%	16000
	rollout		4608	100%	16000
	trajectory	✓	$\sim 1727^{\dagger}$	62.5%	~ 10000
Walker2d-v5	rollout	✓	4718	100%	16000
	rollout		355	100%	16000
	trajectory	✓	$\sim 4713^{\dagger}$	70.8%	~ 6225

2 代码补全

2.1 collect_trajectories 实现 (对应作业 1.1.1–1.1.5)

普通 rollout 按固定步数截断: 每个 epoch 中每个并行环境交互满 `interact_per_epoch` 步就返回, 不等 episode 结束。`collect_trajectories` 改用轨迹条数截断, 外层循环直到 `TrajectoryRollout` 存满 `trajnum` 条轨迹才停止, 每条轨迹以环境 `done` 或达到 `max_episode_length` 结束。

每条轨迹需要保存 `states`、`actions`、`rewards`、`done`s、`log_probs`：`states` 让 actor 重算 $\log \pi_\theta(a|s)$ 也让 critic 估计 V ；`actions` 与 `states` 配合算重要性采样比率；`rewards` 用于反向递推 GAE；`done`s 在递推时乘以 $(1 - d_t)$ 切断 episode 边界的 bootstrap；`log_probs` 是旧策略概率，用来构造 $r_t(\theta) = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})$ 。环境终止时调用 `add_traj()` 保存当前轨迹并重置该 env 的缓冲区；未填满 `max_episode_length` 的位置保持零初始化，`masks` 置 0，后续用 `masks==1` 过滤掉 padding。

`masks` 在这里是关键变量：TrajectoryRollout 预分配形状是 $(\text{trajnum}, \text{max_episode_length})$ ，短轨迹后必有零填充，`add_traj()` 中用 `self.masks[pos,:traj_len]=1` 记录真实长度。如果不 mask 就直接计算，padding 的零奖励会污染 GAE 和 advantage 的均值方差，归一化随之偏移，样本利用率统计也会虚高。此外，收集完一批轨迹后必须重新初始化环境——否则某些 env 会停在 episode 中途，下一轮 trajectory 从非起点状态开始，“破坏”从初始分布出发”的假设。这两点都不影响 TRPO 的 on-policy 性质，因为数据始终来自当前策略，更新一次后即清空。

2.2 _update_policy 实现 (对应作业 1.2.1–1.2.4)

TRPO 的核心理论依赖如下定理：

Theorem 1 (Schulman et al. 2015)

令 $\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$ ， $\epsilon = \max_{s,a} |A_{\pi_{\text{old}}}(s,a)|$ ，则下界成立：

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2.$$

2.2.1 定理含义与单调改进

$L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 是固定旧策略状态分布下的代理目标（重要性采样加权 advantage）， $\frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2$ 是新旧策略 TV 距离引起的惩罚项，其中 $\epsilon = \max_{s,a} |A_{\pi}(s,a)|$ 是 advantage 的绝对值上界， $\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$ 是新旧策略最大 TV 距离。相比于 Kakade–Langford 早期只对混合策略 $\pi_{\text{new}} = (1 - \alpha)\pi_{\text{old}} + \alpha\pi'$ 成立的结论，TRPO 这里把 α 直接定义成最大 TV 距离，便能覆盖任意随机策略（包括神经网络高斯策略）。

只要代理目标提升量超过惩罚项，即 $L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2 \geq \eta(\pi_{\text{old}})$ ，下界上升，策略性能单调改进。这里有个关键的几何直觉：收益项 $L - \eta(\pi_{\text{old}})$ 相对于策略偏离量 α 是一次的，而惩罚项 $C\alpha^2$ 是二次的。在 α 接近 0 时，二次项比一次项小一个数量级，所以一定存在一个小的 α 让收益压过惩罚——这也是为什么”步子迈得够小就一定能改进”在数学上是有保证的。TRPO 的设计思路就是顺着这个直觉走：最大化 L （鼓励 advantage 为正的動作）同时约束策略变化幅度，即在”信赖域”内寻找最优更新：

$$\max_{\pi_{\text{new}}} L_{\pi_{\text{old}}}(\pi_{\text{new}}) \quad \text{s.t.} \quad D_{\text{KL}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}}) \leq \delta.$$

这个定理与其说决定了 TRPO 实际能跑多好，不如说提供了它的设计坐标系。”步长足够小就能局部改进”这一直觉，普通 policy gradient 在精确梯度下也有；真正困难的是深度 RL 里梯度是有限样本估计、策略非线性、advantage 方差大，步长过小学不动、过大采样分布就漂走了。TRPO 的关键在于把”不要离旧策略太远”变成可执行的更新规则——用 KL 量距离，用 Fisher 矩阵修正方向，用共轭梯度和 line search 落到神经网络上。

2.2.2 Surrogate Objective 的实际形式

代理目标的理论形式 $L_{\pi_{\text{old}}}(\pi_{\text{new}}) = \eta(\pi_{\text{old}}) + \sum_s d^{\pi_{\text{old}}}(s) \sum_a \pi_{\text{new}}(a|s) A_{\pi_{\text{old}}}(s, a)$ 依赖真实的旧策略状态分布 $d^{\pi_{\text{old}}}$ ，无法精确得到。工程上用旧策略采样数据做 Monte Carlo 估计，并通过重要性采样修正：

$$\hat{L}(\theta) = \frac{1}{N} \sum_{t=1}^N \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t,$$

$\hat{A}_t$ 由 GAE 估计。代码中先保存采样时的 `old_log_probs`，更新时计算 `ratio = exp(new_log_probs - old_log_probs)`，最后 `surrogate = (ratio * advantages).mean()`。在 $\theta = \theta_{\text{old}}$ 处 $\hat{L}$ 与 η 的梯度相等（两者相差一个常数 $\eta(\pi_{\text{old}})$ ），所以沿 $\hat{L}$ 的梯度走极小步就等价于沿 η 走。

2.2.3 KL 约束、Fisher 矩阵与线搜索

TRPO 实际约束的是 KL 而非定理中的 TV。由 Pinsker 不等式 $D_{\text{TV}}(p, q) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(p||q)}$ ，约束 KL 就能隐式控制 TV，且约束更紧；KL 对高斯策略有闭式且处处可微，TV 在连续空间不光滑、难以做梯度和二阶近似。在当前参数邻域，对 KL 做二阶泰勒展开：

$$D_{\text{KL}}(\theta_{\text{old}}, \theta) \approx \frac{1}{2} \Delta \theta^{\top} F \Delta \theta, \quad F = \nabla_{\theta}^2 D_{\text{KL}}(\theta_{\text{old}}, \theta) \Big|_{\theta=\theta_{\text{old}}} = \mathbb{E} \left[\nabla_{\theta} \log \pi_{\theta} \nabla_{\theta} \log \pi_{\theta}^{\top} \right] \Big|_{\theta_{\text{old}}}$$

其中 F 是 Fisher 信息矩阵，信赖域约束因此化为椭球约束 $\frac{1}{2} \Delta \theta^{\top} F \Delta \theta \leq \delta$ 。线性化 surrogate 后的最优方向就是自然梯度 $\Delta \theta_{\text{base}} = F^{-1} g$ ——这其实就是自然策略梯度 (NPG) 的思路 [4, 3]：不在参数欧氏距离下走普通梯度，而是在策略分布的 KL 几何下修正方向；后续 MDPO 等工作也把 TRPO、PPO 放在 mirror descent 框架下统一比较 [5]。满足约束的最大步长为 $\Delta \theta^* = \sqrt{2\delta / (\Delta \theta_{\text{base}}^{\top} F \Delta \theta_{\text{base}})} \cdot \Delta \theta_{\text{base}}$ 。

神经网络参数太多，显式构造并求逆 F 不现实。代码中只实现 Hessian-vector product：`kl_product(v)` 计算 Fv ，再用共轭梯度近似解 $Fx = g$ ，得到 $x \approx F^{-1}g$ ；实现里 `return product + vector * 0.1` 相当于给 F 加阻尼 λI 改善条件数。最后还要做回溯线搜索：沿 $\theta_k = \theta_{\text{old}} + \beta^k \Delta \theta^*$ 逐步缩小步长，直到实际 $\text{KL} \leq \delta$ 且 surrogate 不下降；两个条件缺一不可，全部候选失败则回滚参数。

3 探索部分

3.1 不同环境的训练难度 (对应作业 2.1)

三个环境的难度差异很明显：rollout 末期平均回报分别约为 Humanoid 1083、HalfCheetah 4276、Walker2d 4718。HalfCheetah 和 Walker2d 的曲线持续上升并收敛到较高水平；Humanoid 因为 348 维状态和复杂的身体协调要求，提升慢、最终值也低，同样超参数下很难追上。

3.2 Advantage 与 Return 的计算差异 (对应作业 2.3)

两种模式都使用 GAE: $\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t)$, $A_t = \delta_t + \gamma\lambda(1 - d_t)A_{t+1}$, $R_t = A_t + V(s_t)$ 。区别是：trajectory 对每条完整轨迹反向递推、用 masks 去 padding, episode 边界更精确；rollout 对固定长度片段反向递推，所有 transition 都参与更新，但若片段末端不是自然终止，把末端 value 当作 0 就会引入 truncation bias。本项目两种模式都把最后一步 next value 置零，这对自然终止的 trajectory 合理，对截断的 rollout 不完全合理。这一差异部分解释了为什么 trajectory 在高维任务中有时表现更好。

3.3 Advantage Normalization 的影响 (对应作业 3.1.1–3.1.3)

固定 rollout 模式、只切换 advan_norm，三个环境差异很大（图 1）：HalfCheetah 不归一化反而更高（4608 vs 4276），Humanoid 两组几乎一样（1085 vs 1083），Walker2d 完全相反——归一化组达到 4718，不归一化长期停留在 355，基本没学到。

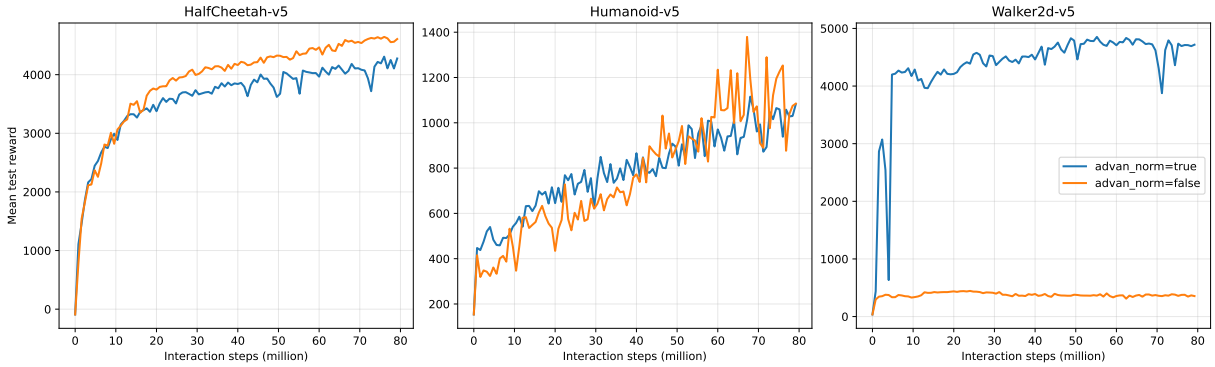


Figure 1: 三个环境中 advantage normalization 对测试平均回报曲线的影响

Walker2d 的稳定性指标更能说明问题：不归一化时 surrogate 只有 86.6% 的记录为有限值，有限值最大绝对值约 9.83×10^{33} ；平均 KL 仅 0.00213，约 4% 的更新中 KL 接近 0，说明大量更新退化或失败。归一化后 surrogate 全部有限，平均 KL 为 0.00881，能稳定利用 $\delta = 0.01$ 的信赖域。Humanoid 中，归一化组最后十次回报标准差为 57.1，不归一化组为 123.8（图 2）。

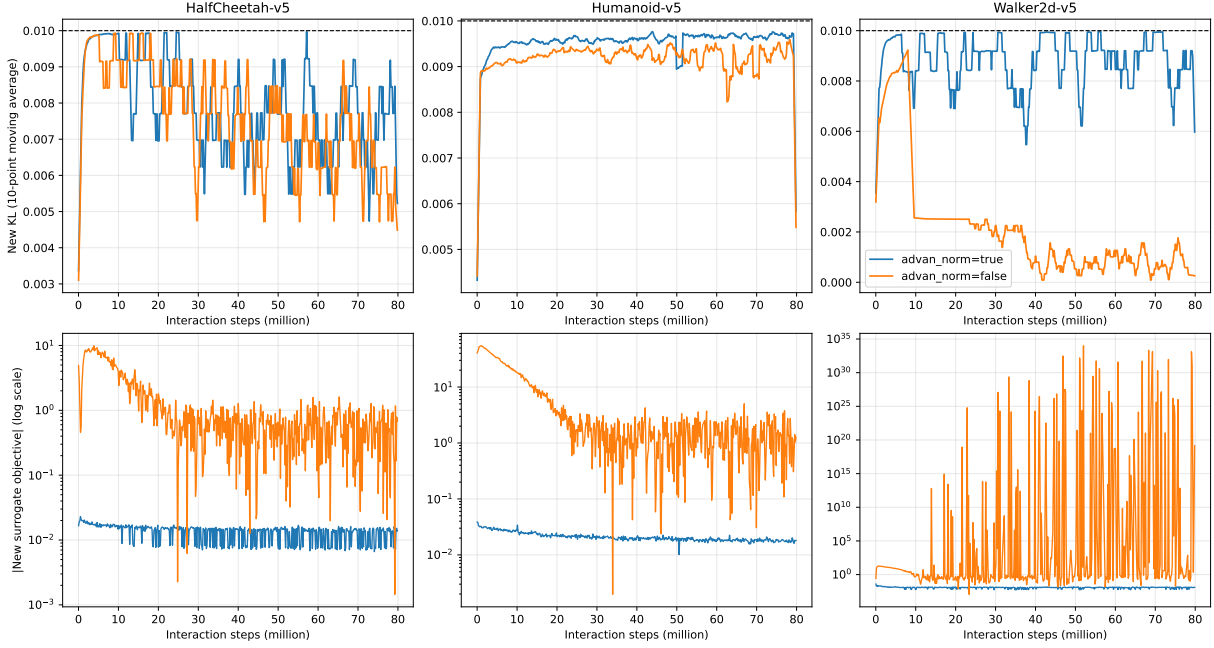


Figure 2: Advantage normalization 对 KL divergence 与 surrogate objective 的影响

所以归一化更像是数值鲁棒性的兜底——TRPO 已经用 KL 约束限制了更新幅度，advantage 尺度本来稳定的环境（HalfCheetah），减去 batch 均值反而改变了有限样本下的梯度方向、引入统计噪声；但一旦 advantage 量级失控（Walker2d），归一化就是必须。

3.4 Trajectory 与 Rollout 的样本利用率（对应作业 2.2、3.2.1–3.2.3）

rollout 配置 8 个并行环境、`interact_per_epoch=2000`，每次更新固定使用 $8 \times 2000 = 16000$ 个 transition；trajectory 每次收集 10 条轨迹，有效步数由 `masks.sum()` 决定。5000 次更新平均下来，HalfCheetah、Humanoid、Walker2d 实际使用 transition 约 10000、885、6225，对应利用率约 62.5%、69.0%、70.8%（图 3）。HalfCheetah 差距最大，原因是 episode 通常到 1000 步上限、并行环境结束时间高度同步，存满 10 条后剩余几条整批被丢弃。

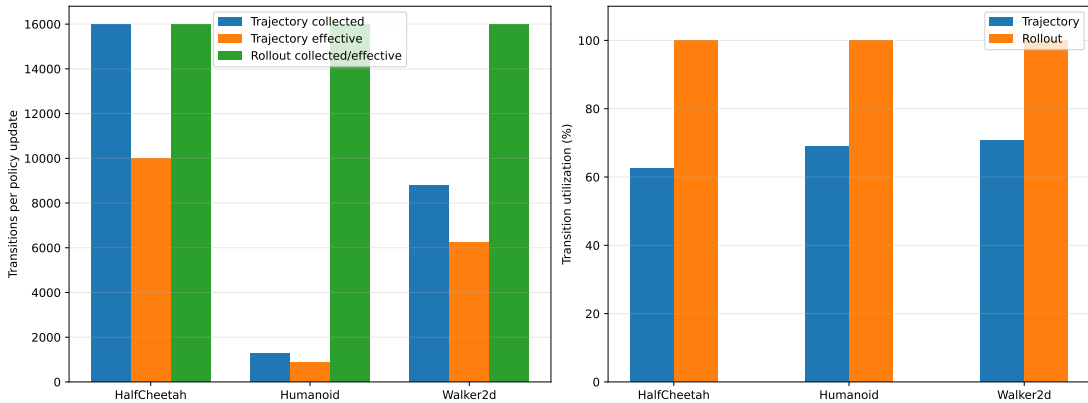


Figure 3: Trajectory 与 rollout 每次更新的 transition 数量与样本利用率

但利用率高不等于效果好。相同交互步数处，HalfCheetah rollout (4276) 远好于 trajectory (1727); Humanoid 在 6.34×10^6 步处 trajectory (655) 反而高于 rollout (459); Walker2d 在 4.33×10^7 步处两者接近 (4713 vs 4657)。rollout 不丢样本但有截断偏差，trajectory 浪费样本但 episode 边界干净，是数量与质量的权衡。

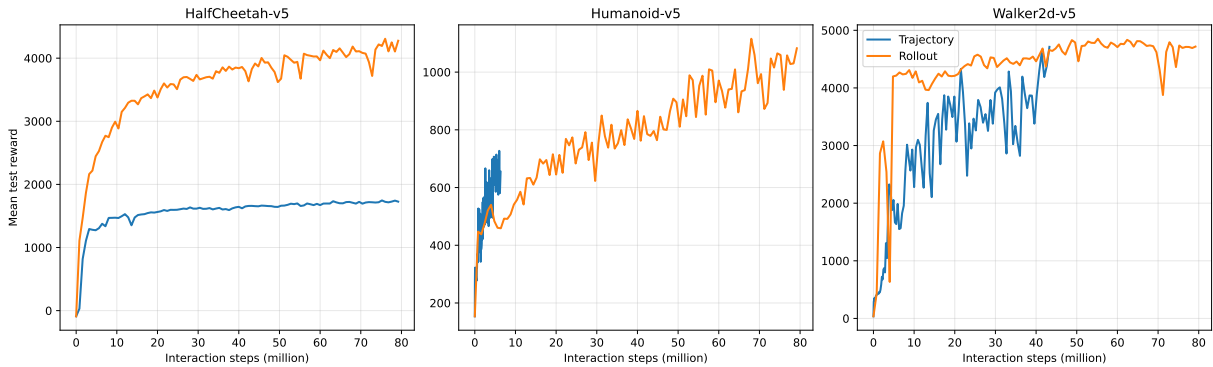


Figure 4: Trajectory 与 rollout 的测试平均回报曲线

3.5 训练过程诊断：critic loss、step size 与 backtrack (诊断补充)

除 reward/KL/surrogate 之外，trpo.py 的 `result.add_metric` 还记录了 critic 的 `value/loss`、回溯线搜索的 `actor/backtrack_step`、自然梯度方向的步长 `actor/stepsize`、二次型 `actor/xHx` 与 `gradient_direction_l2norm`。这几个量直接反映 TRPO 更新链路上每一环的健康状况；从 SwanLab/TensorBoard 日志中取出 `value/loss`、`actor/backtrack_step`、`actor/stepsize` 绘制成图 5，并把六组配置的关键统计汇总在表 3。

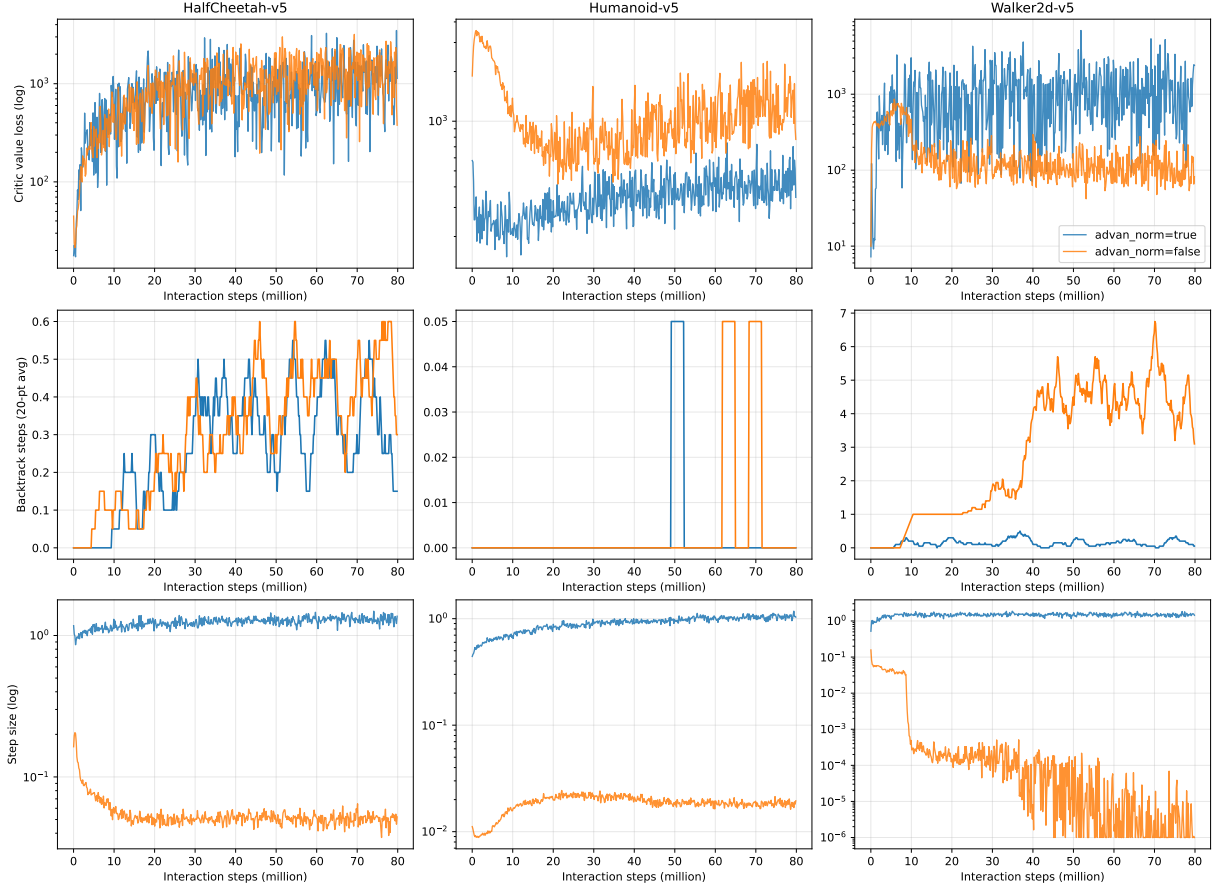


Figure 5: 三个环境、advan_norm true/false 下的 critic value loss (log 轴)、回溯步数 (20-pt 滑动平均) 与自然梯度步长 (log 轴)。

Table 3: 诊断指标统计 (rollout 配置)

环境	advan_norm	value loss 均值	回溯步均值	回溯步最大	step size 均值
HalfCheetah	true	903.9	0.26	1	1.24
HalfCheetah	false	1018.6	0.30	1	5.54×10^{-2}
Humanoid	true	349.1	0.00	1	0.90
Humanoid	false	1130.2	0.00	1	1.85×10^{-2}
Walker2d	true	1093.8	0.14	1	1.47
Walker2d	false	170.3	2.86	9	5.34×10^{-3}

Critic value loss. HalfCheetah、Walker2d 的 critic loss 都呈典型的”先快速下降、再随策略改进缓慢爬升”的形态——策略变好后 return 量级上涨，MSE 目标随之放大；归一化与否的曲线大致重合。Humanoid 是一个反例：不归一化时 critic loss 均值从 349 升到 1130 (约 3.2 倍)，而 reward 几乎相同——说明 critic 在拟合一个被未归一化 advantage 间接放大的 return 目标，但 actor 因步长太小 (见下) 无法把这些信号转化为策略改进。Walker2d 不归一化时 critic loss 反而看起来更低 (170)，原因不是拟合得好，而是 reward 长期停留在 355、return 量级本来就小，并不能反推出 critic 健康。

Step size 与 backtrack. TRPO 每次更新先解 $Fx = g$ 得到方向 $\Delta\theta_{\text{base}}$, 按 $\sqrt{2\delta/(\Delta\theta_{\text{base}}^\top F \Delta\theta_{\text{base}})}$ 给出理论最大 stepsize, 再做最多 `backtracking_steps` 次回溯。两条诊断量一起看出三件事:

- (1) **二阶近似多数情形下是可信的:** HalfCheetah、Humanoid、Walker2d 三个归一化配置的回溯均值都在 $0 \sim 0.3$, 最大也只到 1——CG 解出来的方向第一次试探基本就被采纳。
- (2) **不归一化让”步子根本迈不出去”:** 三个不归一化配置的 step size 都比对应归一化组小 $1 \sim 2$ 个数量级 (HalfCheetah $1.24 \rightarrow 0.055$ 、Humanoid $0.90 \rightarrow 0.019$ 、Walker2d $1.47 \rightarrow 0.0053$)。原因是 advantage 未中心化时 $\Delta\theta_{\text{base}}^\top F \Delta\theta_{\text{base}}$ 被放大, 按 $\sqrt{2\delta/\cdot}$ 反推的步长自然被压缩; 这与 §3.3 给出的 Walker2d 平均 KL 0.00213 远小于 $\delta = 0.01$ 完全自洽——根本没用满信赖域。
- (3) **Walker2d 不归一化是唯一回溯频繁失败的配置:** 均值 2.86、最高 9 次 (达到 `backtracking_steps` 上限即全部失败、参数回滚)。结合 §3.3 的 surrogate 量级 10^{33} , 这就是 advantage 尺度爆掉后 line search 反复检查不通过的直接表现。

综合 §3.3 与本节诊断: advantage normalization 的作用不是”提升 reward”, 而是把整个 TRPO 更新链路 (advantage→surrogate→CG 方向 →step size→line search) 控制在数值合理的区间内; 一旦关闭, 链路上任何一环都可能崩——Walker2d 是 line search 崩, Humanoid 是 step size 被压到几乎不动, HalfCheetah 因为环境本身 advantage 尺度稳定, 所以暴露不出问题。

4 反思

`collect_trajectories` 不会改变 TRPO 的 on-policy 性质: 两种模式都用当前策略采数据、更新一次后清空缓冲区, 区别只是停止条件和数据组织形式。如果只能保留一种采样方式, 我会选 rollout——它每次固定用满 16000 个 transition, 批量稳定、实现简单, HalfCheetah 上明显更好, Walker2d 上最终性能也很好。trajectory 在 Humanoid 等高维任务相同步数下偶尔会高一些, 但综合下来 rollout 更稳定。如果继续做, 给 rollout 末端加上正确的 value bootstrap 能进一步缩小截断偏差。

写完报告后我对 TRPO 的理解也比一开始更具体。最初我只把它记成”用 KL 限制策略更新”, 单调改进定理是它的理论外壳, 但这套推导依赖精确期望、精确 advantage 和理想化的约束求解——一旦进入神经网络和有限样本, 理论本身并不能直接回答”为什么 vanilla policy gradient 不行、TRPO 就行”。普通梯度法配上足够小的步长也有局部上升的直觉, 问题在于这个步长往往小到训练效率很差, 而高方差采样让”足够小”很难判断。所以我现在更愿意把 TRPO 看成是 natural policy gradient / mirror descent 的工程化版本: 用 KL 和 Fisher 矩阵重新度量”一步走多远”, 再用 Hessian-vector product、共轭梯度、line search 让这件事在大网络上能算出来。

这次实验也让我对 implementation choice 的影响有了更直观的感受。Walker2d 关掉 advantage normalization 之后 surrogate 直接出现非有限值, 一个看似只是标准化

的细节就能决定能不能训。同样地，trajectory 的 `masks`、rollout 末端是否 bootstrap、buffer 满时 `pos` 回到 0 这些都不写在论文主公式里，但都会直接影响 advantage、KL 和梯度的可信度。这一点和 [6] 的观察一致：深度 RL 里代码级细节会显著左右算法表现。我卡得最久的几次都不是公式本身，而是 `ReplayBuffer.pos` 在缓冲区满后回到 0、`masks` 没用对、tensor shape 不一致这类问题。如果项目能加几个针对 advantage 计算、满 buffer 和 trajectory mask 的小单元测试，应该能省下不少排查时间；README 也建议补一组能快速验证的短实验命令和 SwanLab 不可用时的 TensorBoard 备用方案。

5 总体结论

- (1) 三个环境难度差异明显：HalfCheetah 最易收敛 (4276)，Walker2d 最高 (4718)，Humanoid 因高维状态最难 (1083)。
- (2) rollout 样本利用率 (100%) 高于 trajectory (62% ~ 71%)，但 Humanoid 和 Walker2d 在相同交互步数下 trajectory 反而略好——利用率高不等于训练效果好。
- (3) advantage normalization 是数值稳定性的兜底：Walker2d 关掉后 surrogate 出现 10^{33} 量级的非有限值；但 HalfCheetah 不归一化反而更高，说明效果依赖环境。
- (4) TRPO 的理论给出单调改进的设计原则，但实际表现还取决于 advantage normalization、rollout 末端 bootstrap、CG 阻尼等实现细节。可以把 TRPO 视为 NPG / mirror descent 的工程化版本，理论和实现两侧需要一起看。

附录 A 问题应答索引

为便于对照作业要求，下表列出每个 README 提问对应在本报告中的位置与一句话回答。

题号	问题	位置	简答
1.1.1	<code>collect_trajectories</code> 与 普通 rollout 的停止条件	§2.1 第 1 段	rollout 按固定 <code>interact_per_epoch</code> 步截断; trajectory 按”存满 <code>trajnum</code> 条完整轨迹”截断, 单条轨迹以 <code>done</code> 或达到 <code>max_episode_length</code> 结束。
1.1.2	每条轨迹需保存什么信息, 分别用在哪一步	§2.1 第 2 段	保存 <code>states</code> / <code>actions</code> / <code>rewards</code> / <code>dones</code> / <code>log_probs</code> : <code>states</code> 喂 actor 与 critic, <code>actions</code> 配 ratio, <code>rewards</code> 反推 GAE, <code>dones</code> 切断 bootstrap, <code>log_probs</code> 是 π_{old} 用于重要性采样。
1.1.3	终止信号下轨迹如何结束, 未填满部分如何处理	§2.1 第 2 段	<code>add_traj()</code> 落盘并重置该 env 缓冲区; 未填满位置零初始化、 <code>masks=0</code> , 下游用 <code>masks==1</code> 过滤。
1.1.4	<code>masks</code> 的作用; 不使用会怎样	§2.1 第 3 段	区分真实 transition 与 padding; 不用 mask 会让零 reward 污染 GAE、advantage 均值方差偏移、归一化失真、样本利用率虚高。
1.1.5	收集完一批轨迹后为何重新初始化环境	§2.1 第 3 段	否则部分 env 停在 episode 中段, 下一批起点不来自初始分布, 破坏从 ρ_0 出发的假设; 这不影响 on-policy 性质。
1.2.1	Theorem 1 每项含义与单调改进条件	§2.2.1 全节	L 是旧分布下的代理目标, $\frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2$ 是 TV 惩罚; 只要 $L - C\alpha^2 \geq \eta(\pi_{old})$, 下界上升, 单调改进。
1.2.2	与 Kakade–Langford 的 Theorem 4.1 处理上的不同	§2.2.1 第 1 段	Kakade–Langford 只对混合策略 $\pi_{new} = (1 - \alpha)\pi_{old} + \alpha\pi'$ 成立, α 是显式混合系数; TRPO 将 α 直接定义为 D_{TV}^{max} , 可适用于任意随机策略 (含高斯神经网络策略)。
1.2.3	surrogate objective 实际怎么写	§2.2.2 全节	$\hat{L}(\theta) = \frac{1}{N} \sum_t \frac{\pi_\theta(a_t s_t)}{\pi_{\theta_{old}}(a_t s_t)} \hat{A}_t$; 代码中 <code>ratio=exp(new_logp-old_logp)</code> , <code>surrogate = (ratio*adv).mean()</code> 。
1.2.4	为何约束 KL 而非 TV; 为何 line search	§2.2.3 全节	Pinsker 不等式让 KL 隐式控制 TV 且约束更紧; KL 对高斯策略闭式可微、可做二阶展开得到 Fisher。线搜索是因 surrogate 一阶近似、KL 二阶近似都是近似, 需要在真参数上检查 $KL \leq \delta$ 且 surrogate 提升。
2.1	三个环境训练难度是否一致	§3.1 全节	不一致: HalfCheetah/Walker2d 持续上升收敛到 4000+, Humanoid 因 348 维状态与协调难度提升慢、最终值低。
2.2	哪个环境样本利用率差异最大	§3.4 第 1 段	HalfCheetah 差距最大 (62.5%); episode 常达 1000 上限、并行环境同步结束, 存满 10 条后剩余整批丢弃。
2.3	advantage / return 计算两种模式	§3.2 全	都用 GAE。trajectory 用 <code>masks</code> 在每条

References

- [1] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning* (ICML 2015), pp. 1889–1897. PMLR.
- [2] Kakade, S., & Langford, J. (2002). Approximately Optimal Approximate Reinforcement Learning. In *Proceedings of the 19th International Conference on Machine Learning* (ICML 2002), pp. 267–274.
- [3] OpenAI. Spinning Up: TRPO. <https://spinningup.openai.com/en/latest/algorithms/trpo.html>
- [4] Kakade, S. M. (2001). A Natural Policy Gradient. In *Advances in Neural Information Processing Systems 14*.
- [5] Tomar, M., Shani, L., Efroni, Y., & Ghavamzadeh, M. (2020). Mirror Descent Policy Optimization. *arXiv preprint arXiv:2005.09814*.
- [6] Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., & Madry, A. (2020). Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO. *arXiv preprint arXiv:2005.12729*.